

Pydantic Library Notes

Prepared By:

M.Shahjhan Gondal



-Errors and omissions excepted -

Contents

1. Python Basics

- Dynamic typing
- Type hints

2. Validation Issues

- Manual validation problems
- Redundant code

3. Pydantic Intro

- Benefits
- Workflow

4. Basic Models

- Field definitions
- Error handling

5. Field Types

- Lists, Dicts, Optional
- Default values

6. Built-in Validation

- Email, URL checks

7. Custom Validation

- Field constraints
- Examples

8. Metadata

- Field docs
- Annotated

9. Strict Mode

- No type coercion

10. Field Validators

- Custom rules

11. Model Validators

- Cross-field checks

12. Computed Fields

- Auto calculations

13. Nested Models

- Model reuse

14. Serialization

- Dict/JSON export

15. Advanced

- Partial updates

Simple Python Data variable with data types:

In Python, variables do not have a fixed data type by default. Python uses dynamic typing, which means:

- You don't need to declare the data type of a variable explicitly.
- A variable's data type is determined automatically at runtime based on the assigned value.
- You can change the data type of a variable simply by assigning it a different type of value.

```
def patient_data(name,age):  
    print(name)  
    print(age)
```

```
patient_data(name="John", age='thirty')
```

output:

```
John  
thirty
```

```
patient_data(name="John", age=30) |
```

output:

```
John  
30
```

Methods for Static Data Type

Type Hinting:

It is used just to inform about type of data variable ,but it cannot raise error if data type changes.

- Type hints don't enforce types
- Not reliable for strict type safety

Not god way to enforce users to deal with correct data type.

Applying Type Hinting 'str' & 'int'



```
def patient_data(name: str , age:int):  
    print(name)  
    print(age)
```

```
patient_data(name="John", age='thirty')
```

output:

```
John  
thirty
```

```
patient_data(name="John", age=30)
```

output:

```
John  
30
```

Problem:

The Output is same as before but
Developers get the correct data type of the variable

```
(parameter) name: str  
patient_data(name="John", age=30)
```

```
(parameter) age: int  
patient_data(name="John", age=30)
```

Conditions:

we use another way ,if-condition for data validation

```
def patient_data(name: str , age:int):  
    if type(name)==str and type(age)==int:  
        if age < 0 :  
            raise ValueError("Age cannot be negative.")  
        else:  
            print(name)  
            print(age)  
            print("Patient data is valid")  
    else:  
        raise TypeError("Invalid data types provided for name or age.")
```

Problem:

- if we have multiple parameters ,we have to add more and separate lines for each parameter validation.
- if we have multiple classes like insert,update ,delete, we have to add validation code lines separately in each class

Pydantic Library Models

- Pydantic is powerful python library that used for the Data validation & Manipulation
- especially used in *APIs* or *user Inputs*
- this makes the code more *Robust,Readable,Concise,easier to Debug*

Procedure Steps to use Pydantic

1-Define a Pydantic model:

- Represent ideal schema of the data
- this includes the expected fields, their types, any validation

2-Instantiate the model by raw input Data:

- input data: usually dictionary or JSON format
- pydantic automatically validate the data & coerce it into correct python types
- raise ValidationError if data type not correct

3-Pass the Validated model object to functions :

- ensures every part of program works with clean, type-safe , logically valid data

Example: Applying 3 steps of Pydantic Process

```
1
2 from pydantic import BaseModel
3
4 class Patient(BaseModel):          # Define a Patient model with name (str) and age (int)
5     name: str
6     age: int
7
8 patient_info = {'name': 'Gondal', 'age': 30} # Dictionary containing patient information
9
10 patient1 = Patient(** patient_info)        # Create an instance
11
12 def patient_data(patient: Patient):         # Function that accepts a Patient object and prints its data
13
14     print(patient.name)
15     print(patient.age)
16     print("Patient data is valid")
17
18 patient_data(patient1)
19
20
```

output:

```
Gondal
30
Patient data is valid
```

it will raise **ERROR** when data type change **integer ('30')** to **string ('thirty')**

```
2 from pydantic import BaseModel
3
4 class Patient(BaseModel):          # Define a Patient model with name (str) and age (int)
5     name: str
6     age: int
7
8 patient_info = {'name': 'Gondal', 'age': 'thirty'} # Dictionary containing patient information
9
10 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary patient_info = {'name':
```

Exception has occurred: ValidationError X
1 validation error for Patient
age
Input should be a valid integer, unable to parse string as an integer [type=int_parsing, input_value='thirty', input_type=str]
For further information visit https://errors.pydantic.dev/2.11/v/int_parsing
File "C:\Users\Decent computers\Desktop\PyDantic\pydantic-library.py", line 10, in <module>
 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
 ~~~~~  
pydantic\_core.ValidationError: 1 validation error for Patient

**Now we can**

- add more parameters in the Model class and corresponding Dictionary
- we can add more classes that can access data variables with their data -types, no need to write separately parameters and types

**Example:**

- Insert class use two parameters with data types
- update class use three parameters with data types

```
from pydantic import BaseModel

class Patient(BaseModel):          # Define a Patient model with name (str) and age (int)
    name: str
    age: int
    weight: float

patient_info = {'name': 'Gondal', 'age': 30, 'weight': 49.89} # Dictionary containing patient information

patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary

def patient_data_insert(patient: Patient):          # Function that accepts a Patient object and prints its data
    print(patient.name)
    print(patient.age)
    print("Patient data inserted")

patient_data_insert(patient1)

def patient_data_update(patient: Patient):          # Function that accepts a Patient object and prints its data
    print(patient.name)
    print(patient.age)
    print(patient.weight)
    print("Patient data updated")

patient_data_update(patient1)
```

```
Gondal
30
Patient data inserted
Gondal
30
49.89
Patient data updated
```

## Now ,we add more data types and their validation for understanding

### Example:

```
from pydantic import BaseModel
from typing import List, Dict

class Patient(BaseModel):
    # Define a Patient model with name (str) and age (int)
    name: str
    age: int
    weight: float
    marital_status: bool
    allergies: List[str]
    contact_detail: Dict[str, str]

patient_info = {'name': 'Gondal', 'age': 30, 'weight': 49.89, 'marital_status': True,
               'allergies': ['Pollen', 'Dust'], 'contact_detail': {'email': 'abc@gmail.com', 'phone': '+09876431'}} #

patient1 = Patient(** patient_info) # Create an instance to unpack the dictionary

def patient_data_insert(patient: Patient):
    # Function that accepts a Patient object and prints its data
    print(patient.name)
    print(patient.age)
    print(patient.weight)
    print(patient.marital_status)
    print(patient.allergies)
    print(patient.contact_detail)
    print("Patient data inserted")

patient_data_insert(patient1)
```

List[ str ] --- validate that the items of list must be string

Dict[str, str] --- validate that the items of dict must have *key:string* & *value:string*

```
Gondal
30
49.89
True
['Pollen', 'Dust']
{'email': 'abc@gmail.com', 'phone': '+09876431'}
Patient data inserted
```

## Optional & Required fields

- By default, all fields are required
- we can create the Optional fields for user ,using keyword ' **Optional**'
- Optional fields need '**default value** ',that assign when user donot fill the Optional fields

e.g. **Optional[List[str]] = None**



we make '**allergies**' & '**contactdetail**' optional fields

```
from pydantic import BaseModel
from typing import List, Dict, Optional

class Patient(BaseModel):
    # Define a Patient model with name (str) and age (int)
    name: str
    age: int
    weight: float
    marital_status: bool
    allergies: Optional[List[str]] = None # Optional list of allergies
    contact_detail: Optional[Dict[str, str]] = {} # Optional dictionary for contact details

patient_info = {
    'name': 'Gondal', 'age': 30, 'weight': 49.89, 'marital_status': True,
    'contact_detail': {'email': 'abc@gmail.com', 'phone': '+09876431'}
} # Dictionary containing patient info

patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
```

```
Gondal
30
49.89
True
None
{'email': 'abc@gmail.com', 'phone': '+09876431'}
Patient data inserted
```

'**allergies**' is None because we enter nothing in this field as in '**patient\_info**', it got default value '**None**'

## Data Validation in Pydantic

Pydantic automatically validates the format of data based on the field types and constraints you define. It can validate:

- Email addresses (e.g., using `EmailStr`)
- Phone numbers (with custom `regex` or third-party libraries)
- URLs (e.g., using `AnyUrl` or `HttpUrl`)

In below Example, we add two validations `Email` and `URLs`

- import `'EmailStr'` and `'AnyUrl'`

- use validations in code

email:`EmailStr`

linkedin\_url:`AnyUrl`

```
from pydantic import BaseModel, EmailStr, AnyUrl
from typing import List, Dict, Optional

class Patient(BaseModel):
    # Define a Patient model with name (str) and age (int)
    name: str
    email: EmailStr
    linkedin_url: AnyUrl
    age: int
    weight: float
    marital_status: bool
    allergies: Optional[List[str]] = None  # Optional list of allergies
    contact_detail: Optional[Dict[str, str]] = None

patient_info = {
    'name': 'Gondal',
    'email': 'abc@gmail.com',
    'linkedin_url': 'https://linkedin.com/in/muhammad...hjhgan-gondal-493884311',
    'age': '30',
    'weight': '49.89',
    'marital_status': True,
    'contact_detail': {'phone': '+09876431'}  # Dictionary containing patient information
}

patient1 = Patient(**patient_info)  # Create an instance to unpack the dictionary
```

```
Gondal
abc@gmail.com
https://linkedin.com/in/muhammad...hjhgan-gondal-493884311
30
49.89
True
None
{'phone': '+09876431'}
Patient data inserted
```

**Note:** it will raise error if we remove '@' from Email or 'https' from the Url

## Custom Data Validation

- this validation can be done according to our Desired fields
- we can apply specific validation rules to different fields based on the type of data we expect.
- we can adjust validation constraints according to our Business needs
- keyword '**Field**' is used for this validation

in below Example ,we apply custom validation on the **name, age ,weight, allergies**

- **name**: str = Field(min\_length=1, max\_length=50) [ Name must be between 1 and 50 characters ]
- **age** : int = Field(gt= 18) [ Age must be greater than 18 and less than 100 ]
- **weight**: float = Field(gt= 35 , lt= 120) [ Weight must be greater than 35 and less than 200]
- **allergies**:Optional[List[str]] = Field(max\_length= 10) [ String name must be less than 10 ]

```
from pydantic import BaseModel, EmailStr, AnyUrl, Field
from typing import List, Dict, Optional

class Patient(BaseModel):
    # Define a Patient model with name (str) and age (int)
    name: str = Field(min_length=1, max_length=50) # Name must be between 1 and 50 characters
    email: EmailStr
    linkedin_url: AnyUrl
    age : int = Field(gt= 18) # Age must be greater than 18 and less than 100
    weight: float = Field(gt= 35 , lt= 120) # Weight must be greater than 35 and less than 200
    marital_status: bool
    allergies: List[str] = Field(max_length= 10) # Optional list of allergies
    contact_detail: Dict[str, str] = None

patient_info = {
    'name': 'Gondal',
    'email': 'abc@gmail.com',
    'linkedin_url': 'https://linkedin.com/in/muhammad...hjhah-gondal-493884311',
    'age': '30',
    'weight': '49',
    'marital_status': True,
    'allergies': ['pollen', 'dust'],
    'contact_detail': { 'phone': '+09876431' }
} # Dictionary containing patient information

patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
```

### int/float

| Constraints | Applies To | Description                                            |
|-------------|------------|--------------------------------------------------------|
| gt          | int/float  | Value must be ' greater than ' this number             |
| ge          | int/float  | Value must be ' greater than or equal to ' this number |
| lt          | int/float  | Value must be ' less than ' this number                |
| le          | int/float  | Value must be ' less than or equal to 'this number     |
| multiple_of | int/float  | Value must be a multiple of the given number           |

### string

| Constraints | Applies To | Description                             |
|-------------|------------|-----------------------------------------|
| min_length  | string     | Minimum number of characters            |
| max_length  | string     | Maximum number of characters            |
| regex       | string     | Must match the given regular expression |

### list

| Constraints | Applies To | Description             |
|-------------|------------|-------------------------|
| min_items   | list       | Minimum number of items |
| max_items   | list       | Maximum number of items |

## Metadata in Pydantic

- Metadata is Additional/extra information added to fields using `Field( . . )`, and to models using `model_config` or `Config(v1)`.
- common Metadata fields in `Field()`

### Parameter

### Purpose

|                                |                                           |
|--------------------------------|-------------------------------------------|
| <code>title</code>             | Short name for the field                  |
| <code>description</code>       | Longer explanation                        |
| <code>example</code>           | Example value shown in docs               |
| <code>alias</code>             | Alternate name for parsing                |
| <code>deprecated</code>        | Mark field as deprecated                  |
| <code>const</code>             | Value must exactly match the default      |
| <code>json_schema_extra</code> | Add arbitrary metadata to the JSON schema |

## Annotated

Annotated is used to attach metadata (like Field constraints) directly to type hints.

### Example:

```
from pydantic import BaseModel, EmailStr, AnyUrl, Field
from typing import List, Dict, Optional, Annotated

class Patient(BaseModel):
    # Define a Patient model with name (str)
    name: Annotated[str, Field(max_length = 50,
                               title = 'Name of Patient',
                               description = 'Give name of Patient less than 50 characters ',
                               examples = ['Ali Gondal', 'Nawab Shah'])] # Name must be between 1 and 50 c
    email: EmailStr
    linkedin_url: AnyUrl
    age: int = Field(gt= 18) # Age must be greater than 18 and less t
    weight: float = Field(gt= 35 , lt= 120) # Weight must be greater than 35 and les
    marital_status: Annotated[bool, Field(default = None,
                                         title = 'Marital Status',
                                         description = 'True if married, False if not married')] # Marital status is
    allergies: Annotated[Optional[List[str]], Field(default = None ,
                                                    max_length = 10 ,
                                                    title = 'Name of Allergies ')]
    contact_detail: Dict[str, str] = None
```

## 'Strict' keyword

- By default, Pydantic is forgiving — it tries to coerce values to the expected types.
- But if you want to enforce that the input is exactly the type you declare (e.g., no converting "123" to 123), use `strict=True`.

```
weight: Annotated[float, Field(gt = 35 ,  
                                lt = 120,  
                                strict=True )]
```

**'Strict' will raise Error if we have following condition:**

```
patient_info = {'name': 'Gondal',  
                'email': 'abc@gmail.com',  
                'linkedin_url': 'https://linkedin.com/in/muhammad...hghan-gondal-493884311',  
                'age': '30',  
                'weight': '49.7',  
                'marital_status': True,  
                'allergies': ['pollen', 'dust'],  
                'contact_detail': {'phone': '+09876431'}} # Dictionary containing patient information
```

```
Exception has occurred: ValidationError ×  
1 validation error for Patient  
weight  
  Input should be a valid number [type=float_type, input_value='49.7', input_type=str]  
  For further information visit https://errors.pydantic.dev/2.11/v/float_type  
File "C:\Users\Decent computers\Desktop\PyDantic\pydantic-libraray.py", line 35, in <module>  
    patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary  
    ~~~~~  
pydantic_core._pydantic_core.ValidationError: 1 validation error for Patient
weight
 Input should be a valid number [type=float_type, input_value='49.7', input_type=str]
```

## Field Validator

- Field validator is used for the **custom validation logic** to individual fields
- It's a decorator used to define custom rules for validating a specific field.
- You use it when built-in options in Field() aren't enough.

### Example: 1

Simple make the Name Uppercase

```
@field_validator('name')
@classmethod
def validate_name(cls, value):
 Uppercase= value.upper()
 return Uppercase
```

### Full code:

```
from pydantic import BaseModel, EmailStr, AnyUrl, Field, field_validator
from typing import List, Dict, Optional, Annotated

class Patient(BaseModel):
 name: str
 email: EmailStr

 @field_validator('name')
 @classmethod
 def validate_name(cls, value):
 Uppercase= value.upper()
 return Uppercase

patient_info = {'name': 'Gondal',
 'email': 'abc@gcuf.com'}

patient1 = Patient(**patient_info)

def patient_data_insert(patient: Patient):
 print(patient.name)
 print(patient.email)
 print("Patient data inserted")

patient_data_insert(patient1)
```

```
GONDAL
abc@gcuf.com
Patient data inserted
PS C:\Users\Decent computers\Desktop\PyDantic>
```

## Example: 2

### Both variables - Name and Email Validation

```
1 from pydantic import BaseModel, EmailStr, AnyUrl, Field, field_validator
2 from typing import List, Dict, Optional, Annotated
3
4
5 class Patient(BaseModel):
6 name: str
7 email: EmailStr
8
9 @field_validator('name')
10 @classmethod
11 def validate_name(cls, value):
12 Uppercase= value.upper()
13 return Uppercase
14
15 @field_validator('email')
16 @classmethod
17 def validate_email(cls, value):
18 valid_domains = ['gcuf.com', 'pucit.com']
19 domain_name = value.split('@')[-1]
20 if domain_name not in valid_domains:
21 raise ValueError(f"Email domain is not valid")
22 return value
23
24 patient_info = {'name': 'Gondal',
25 'email': 'abc@gcuf.com'}
26 # Dictionary containing patient information
27
28 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
29
30 def patient_data_insert(patient: Patient):
31 # Function that accepts a Patient object and prints its data
32 print(patient.name)
33 print(patient.email)
34 print("Patient data inserted")
35
36 patient_data_insert(patient1)
37
38
```

```
GONDAL
abc@gcuf.com
Patient data inserted
```

if i change the mail that is not in valid emails.it will raise **ValueError**

```
24 patient_info = {'name': 'Gondal',
25 'email': 'abc@gmail.com'}
26 # Dictionary containing patient information
27
28 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary patient_info
29
Exception has occurred: ValidationError X
1 validation error for Patient
email
 Value error, Email domain is not valid [type=value_error, input_value='abc@gmail.com', input_type=str]
 For further information visit https://errors.pydantic.dev/2.11/v/value_error
File "C:\Users\Decent computers\Desktop\PyDantic\field_validator.py", line 28, in <module>
 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

## Model Validator

- Model validator is used when need to **validate multiple fields together**
- A special validator for the entire model, not just one field.
- It allows logic that depends on the relationship between fields.
- Runs after all individual fields are validated

### Example:

if we need that over 60 years old Patient must have Emergency contact in his Profile, **model\_validator** is used because two variables relations exist 'age' and 'contact\_detail'

```
@model_validator(mode='after')
def validate_emergency_contact(cls, model):
 if model.age > 60 and 'emergency_contact' not in model.contact_detail:
 raise ValueError("Emergency Contact is required for Patients over 60 years old")
 return model
```

**Model\_validator** will work correctly, if over 60 years Patient has **emergency\_contact**

```
patient_info = {'name': 'Gondal',
 'age': '63',
 'contact_detail': {'phone': '+09876431', 'emergency_contact': '1234567890'},
 'email': 'abc@pucit.com'}
Dictionary containing patient information
```

**Model\_validator** will **Raise Error**, if over 60 patient has **NO emergency\_contact**

```
32
33 patient_info = {'name': 'Gondal',
34 'age': '63',
35 'contact_detail': {'phone': '+09876431'},
36 'email': 'abc@pucit.com'}
37 # Dictionary containing patient information
38
39 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary patient_info

Exception has occurred: ValidationError X
1 validation error for Patient
 Value error, Emergency Contact is required for Patients over 60 years old [type=value_error, input_value={'name': 'Gondal', 'age': '63', 'email': 'abc@pucit.com'}, input_type=dict]
 For further information visit https://errors.pydantic.dev/2.11/v/value_error
File "C:\Users\Decent computers\Desktop\PyDantic\field_validator.py", line 39, in <module>
 patient1 = Patient(**patient_info) # Create an instance to unpack the dictionary
    ~~~~~
pydantic_core._pydantic_core.ValidationError: 1 validation error for Patient
  Value error, Emergency Contact is required for Patients over 60 years old [type=value_error, input_value={'name': 'Gondal',
```

## Computed Fields

- **computed\_field** are calculated Automatically based on other field's values
- these are not entered by user

A computed field:

- Does not exist in the input data
- Is calculated dynamically at runtime

Example:

user enters 'Height' and 'weight' , **computed\_field** calculate the **BMI** , automatically computed by the Model

```
@computed_field
@property
def calculate_bmi(self) -> float:
    if self.height <= 0:
        raise ValueError("Height must be greater than zero")
    bmi = round(self.weight / (self.height ** 2), 2)
    return bmi
```

Full code:

```
1 from pydantic import BaseModel, EmailStr, AnyUrl, Field, field_validator, model_validator, computed_field
2
3 from typing import List, Dict, Optional, Annotated
4
5 class Patient(BaseModel):
6     height: float
7     weight: float
8
9     @computed_field
10    @property
11    def calculate_bmi(self) -> float:
12        if self.height <= 0:
13            raise ValueError("Height must be greater than zero")
14        bmi = round(self.weight / (self.height ** 2), 2)
15        return bmi
16
17 # Example usage
18 patient_info = {
19     'height': 1.75, # meters
20     'weight': 68 # kg
21 }
22
23 patient = Patient(**patient_info)
24 print(f"BMI: {patient.calculate_bmi}")
25
```

```
BMI: 22.2
PS C:\Users\Decent computers\Desktop\PyDantic> []
```

## Nested Models

- using one **Base Model** inside another **Base Model** as a field type.

### Why Use Nested Models?

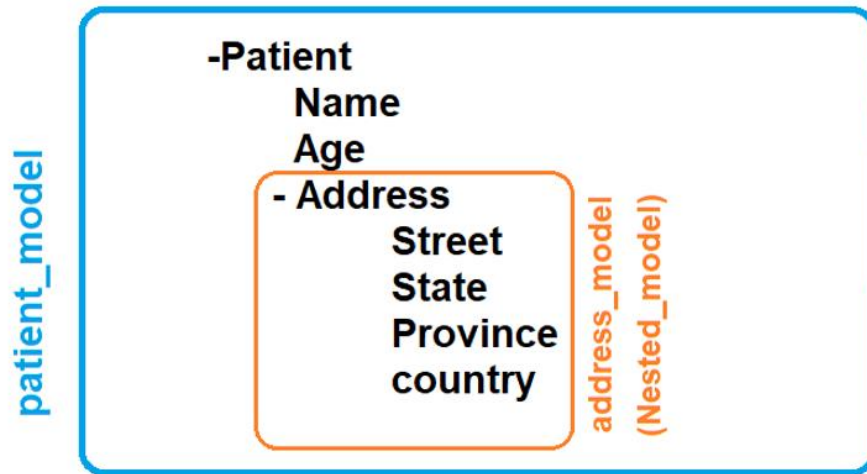
**Clarity:** Break down complex models into manageable parts.

**Reusability:** A nested model like ContactDetail can be reused in multiple places.

**Validation:** Each model validates its own fields independently and cleanly.\

### **Example:**

You are getting *Name, Age, Address* – , it's difficult to get Directly from Address. to get the *Street, State, Province, Country* Separately from Address, we will use Nested\_Model.



### Example:

```
1  from pydantic import BaseModel
2
3  class Address(BaseModel):
4      street: str
5      state: str
6      province: str
7      country: str
8
9  class Patient(BaseModel):
10     name: str
11     age: int
12     address: Address
13
14     # Connected dictionary with matching keys
15     address_dict = {
16         'street': '123 Main St',
17         'state': 'California',
18         'province': 'CA',
19         'country': 'USA'
20     }
21
22     # Create Address model
23     address1 = Address(**address_dict)
24
25     # Now create Patient with the nested address
26     patient_dict = {
27         'name': 'John Doe',
28         'age': 30,
29         'address': address1
30     }
31
32     patient1 = Patient(**patient_dict)
33
34     print(patient1)
35
```

**address\_model**  
**(Nested\_model)**

**patient\_model**

```
name='John Doe' age=30 address=Address(street='123 Main St', state='California', province='CA', country='USA')
```

Now we can extract any part of the Patient and Address

```
36 print(patient1.name)
37 print(patient1.address.country)
```

```
John Doe
USA
```

## Import Models

we can import Models in **Dictionary** , **JSON** format

|                   |    |                          |
|-------------------|----|--------------------------|
| <b>Dictionary</b> | by | <b>model_dump()</b>      |
| <b>JSON</b>       | by | <b>model_dump_json()</b> |

### Dictionary import

```
40 # dictionary import
41 dict_import = patient1.model_dump()
42 print(dict_import)
```

### output

```
{'name': 'John Doe', 'age': 30, 'address': {'street': '123 Main St', 'state': 'California', 'province': 'CA', 'country': 'USA'}}
```

### JSON import

```
44 # JSON import
45 json_import = patient1.model_dump_json()
46 print(json_import)
```

### output

```
{"name": "John Doe", "age": 30, "address": {"street": "123 Main St", "state": "California", "province": "CA", "country": "USA"}}
```

## Include and Exclude

'**include**' and '**exclude**' are used when converting models to dictionaries or JSON, allowing you to **selectively control** which fields appear.

- '**include**' will add mention fields
- '**exclude**' will remove mention fields

### dictionary import with include fields

```
# dictionary import with include
dict_import = patient1.model_dump(include={'name', 'age'})
print(dict_import)
```

```
{'name': 'John Doe', 'age': 30}
```

### dictionary import with exclude fields

```
40 # dictionary import with exclude
41 dict_import = patient1.model_dump(exclude={'age'})
42 print(dict_import)
43
```

```
{'name': 'John Doe', 'address': {'street': '123 Main St', 'state': 'California', 'province': 'CA', 'country': 'USA'}}
```

## exclude\_unset

Only include the fields that were explicitly set during model creation, and exclude those that still have their default values.

### Why Use ?

- To avoid sending unnecessary/default fields in API responses or logs.
- Useful when building partial updates (PATCH) where only set fields should be sent.

```
1 from pydantic import BaseModel, Field
2
3 class User(BaseModel):
4     name: str
5     age: int = 25 # default value
6     email: str = 'not_provided@example.com'
7
8 user = User(name='All')
9 print(user.model_dump(exclude_unset=True))
10
```

```
{'name': 'All'}
```

# Thank You